

🔒 ТЕХНИЧЕСКИЙ ПАСПОРТ И ИБ-АНАЛИЗ МОДУЛЯ АДМИНИСТРИРОВАНИЯ ПАНЕЛИ (panel_users.js)

Настоящий архитектурный документ содержит исчерпывающее описание, пошаговый структурный разбор и профиль информационной безопасности клиентского модуля администрирования аккаунтов самой веб-панели и управления лимитами операторов СУБД (panel_users.js)

МОДУЛЬ АДМИНИСТРИРОВАНИЯ ПАНЕЛИ (panel_users.js)

Контур обеспечивает перехват отправки формы, валидацию типов, динамическое определение режима работы («Создание» / «Модификация») и верный сбор лимитов операторов.

--- Схема рантайм-сбора данных и валидации режима инпута ---

ОТПРАВКА ФОРМЫ (submitRegisterPanelUserForm)

↓ Считывание ОЗУ-инпутов и приведение к нижнему регистру usernameClean

КОНТУР ОПРЕДЕЛЕНИЯ МОДИФИКАЦИИ (isEditButton || isUpdateMode)

Проверка текста кнопки —> Поиск совпадений логина в массиве базы

↓ Разветвление ИБ-проверки заполнения поля пароля

Режим Модификации (finalUpdateCheck == true)

- Допускается пустое поле пароля.
- 🔒 finalPassword = "";
- 🛡️ Защита: Бэкенд обновит только лимиты прав, сохранив старый хэш пароля.

Режим Создания (finalUpdateCheck == false)

- ❌ Пустой пароль категорически запрещен!
- ⚠️ if (!finalPassword) —> Сброс транзакции.
- 🔔 Вызов showAlert('Введите мастер-пароль!').

ПЕРЕХОД К СЕКЦИИ СБОРКИ МАТРИЦЫ ПРАВ ПО РОЛЯМ

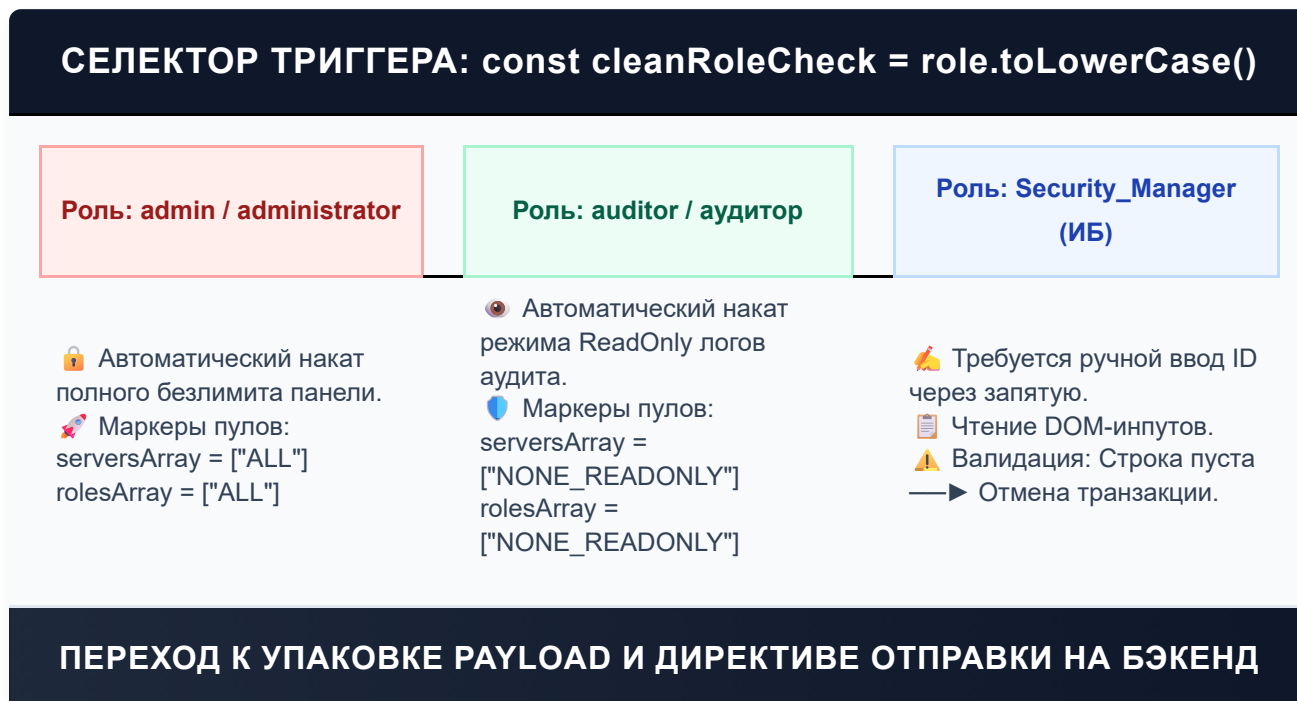
🛡️ Профиль ИБ-защиты подсистемы перехвата формы:

- **Защита от перезаписи и скомпрометирования паролей при апдейтах:** Принудительное разветвление логики на основе детекции finalUpdateCheck гарантирует защиту криптографических хэшей. Если администратор панели изменяет исключительно пул доступных серверов ИБ-офицеру, поле пароля очищается в пустую строку "". Это страхует систему от случайного затирания Bcrypt-хэша пустой строкой на бэкенде.
- **Предотвращение создания пустых или «беспарольных» аккаунтов:** В режиме создания нового пользователя, проверка !finalPassword || finalPassword.length === 0 блокирует трансляцию пакета на сервер. Создание администратора без пароля физически невозможно.

МОДУЛЬ АДМИНИСТРИРОВАНИЯ ПАНЕЛИ (panel_users.js)

Контур обеспечивает каскадную проверку роли создаваемого аккаунта и автоматическую генерацию лимитов серверов и ролей в зависимости от привилегий касты.

--- Схема автоматического распределения пулов лимитов ---



 Профиль ИБ-защиты подсистемы распределения лимитов:

- **Защита от несанкционированного расширения прав (Privilege Escalation):** Код жестко разделяет условия обработки ролей. Если оператор создает учетную запись `admin` или `auditor`, любые текстовые значения, введенные в поля доступных серверов или ролей на экране, полностью игнорируются. Система принудительно перетирает массивы жесткими служебными маркерами `"ALL"` или `"NONE_READONLY"`, исключая возможность внедрения скрытых прав через фальсификацию HTML-инпутов в браузере.
- **Предотвращение обхода лимитов Офицерами ИБ:** Для роли `Security_Manager` заполнение ограничений является критически обязательным. ИБ-затвор `if (!serversRaw || !rolesRaw)` намертво прерывает выполнение функции, если менеджер безопасности не привязал пользователя ни к одному целевому серверу инфраструктуры, предотвращая появление неконтролируемых учетных записей.

МОДУЛЬ АДМИНИСТРИРОВАНИЯ ПАНЕЛИ (panel_users.js)

Контур обеспечивает точную сборку JSON-пакета данных в строгом соответствии со схемой валидации бэкенда и динамически переключает сетевые шлюзы API.

--- Схема упаковки Payload и переключения API-маршрутов ---

СБОРКА ОБЪЕКТА PAYLOAD В ОЗУ БРАУЗЕРА

↓ Спецификация полей: `username`, `password`, `role`, `allowed_servers`, `allowed_target_roles`

ДИНАМИЧЕСКИЙ ТРИГГЕР ВЫБОРА АДРЕСА НАЗНАЧЕНИЯ

`targetApiRoute = finalUpdateCheck ? '/api/admin/update-user-limits' : '/api/admin/configure-panel-manager'`

↓ Разветвление результатов HTTPS-транзакции

Ответ: `response.ok == true`

- Вызов `showAlert(..., 'success')`.
- 🔧 `form.reset()` + Снятие `disabled`-блокировок инпутов.
- 🔄 Мгновенное обновление RAM-кэша: `loadPanelUsers()`.

Ответ: `response.ok == false` / Сбой сети

- ✗ Перехват в секцию `catch(err)`.
- 🔔 Вывод изолированного сообщения об ошибке бэкенда СУБД.
- 🚫 Сохранение старых данных в форме для ручной корректировки.

КОНТУР УСПЕШНО ЗАПЕЧАТАН —► СБРОС ТРИГГЕРОВ КНОПКИ В ДЕФОЛТ

🔒 Профиль ИБ-защиты подсистемы роутинга и упаковки:

- **Строгое соответствие Pydantic v2 на бэкенде:** Структурирование payload-пакета осуществляется явным, ручным присвоением полей. Массивы `serversArray` и `rolesArray` проходят принудительную зачистку пробелов `.map(s => s.trim())` и фильтрацию пустых строк `.filter(Boolean)`. Это исключает отправку мусорных байт, снижая риски возникновения скрытых ошибок синтаксиса СУБД.
- **Автоматический снос блокировок (Anti-Lockup UI):** При получении статуса `response.ok` система не только очищает поля через `form.reset()`, но и принудительно возвращает полю логина свойство `removeAttribute('disabled')`. Это исключает состояние "зависания интерфейса", когда администратор теряет возможность вводить данные после успешной модификации лимитов.

МОДУЛЬ АДМИНИСТРИРОВАНИЯ ПАНЕЛИ (panel_users.js)

Контур обеспечивает чтение реестра администраторов из СУБД, безопасную дедупликацию ОЗУ-кэша и сквозную поисковую фильтрацию строк перед выводом на экран.

--- Схема дедупликации ОЗУ-кэша и схлопывания строк LEFT JOIN ---

ВЫЗОВ ИЗВЛЕЧЕНИЯ ДАННЫХ: `loadPanelUsers()`

↓ Запрос к API периметра: `secureFetch('/api/admin/panel-users-list')`

КОНТУР ДЕДУПЛИКАЦИИ СТРОК (Схлопывание Декартова умножения)

Инициализация структуры `grouped = {}` —► Нарезка уникальных `Set()` по `dbId`



Агрегация массивов допусков: `Array.from(u.serversSet / u.rolesSet)`

Контур А: Поисковый инпут пуст

- `if (filterText === "") return true;`
- 📦 Весь чистый массив ОЗУ-кэша переносится в `globalPanelUsersCache`.
- 🎨 Запуск: `renderPanelUsersTable()`.

Контур Б: Введен текст в строку поиска

- 🔍 Сквозное сравнение `.includes(filterText)`. Сверка по 3 полям: `login`, `role`, `allowed_servers`.
- ✗ Строки без совпадений мгновенно отсекаются из рендеринга.

РЕНДЕРИНГ ОТФИЛЬТРОВАННОЙ ТАБЛИЦЫ ПАНЕЛИ ЗАВЕРШЕН

🔒 Профиль ИБ-защиты подсистемы чтения и дедупликации:

- **Защита от Декартова раздувания ОЗУ-кэша (LEFT JOIN Explosion):** Поскольку бэкенд выгружает связи один-ко-многим через реляционные объединения, на фронтенд может прилетать избыточное количество дублирующихся строк (например, одна строка на каждую комбинацию сервера и роли). Контур дедупликации на строках 239–253 полностью ликвидирует этот риск, схлопывая дубли в уникальные хэш-таблицы объектов `Set()` по числовому первичному ключу `id`.
- **Предотвращение инъекций [object Object] в таблицы:** При разборе массивов допусков внедрена жесткая строковая ИБ-валидация `if (s && s !== "[object Object]")`. Это страхует DOM-дерево интерфейса от аварийного вывода битых JSON-структур в текстовые ячейки.
- **Изолированный поисковый движок:** Функция фильтрации `filterPanelUsersTable` не взаимодействует с опасными методами удаления или перезаписи элементов, она просто считывает инпут и перезапускает `loadPanelUsers()`, защищая кэш от рассинхронизации.

МОДУЛЬ АДМИНИСТРИРОВАНИЯ ПАНЕЛИ (panel_users.js)

Контур обеспечивает постраничный динамический рендеринг таблицы, экранирование строковых переменных перед инжектом и жесткую ИБ-блокировку удаления root-аккаунта.

--- Схема постраничного рендеринга и ИБ-блокировок элементов UI ---

МАССИВ НА РЕНДЕР: `slicedUsers = userList.slice(startIndex, endIndex)`



Итерация по срезу пагинации: `slicedUsers.forEach(r => ...)`

ИБ-ПЕРЕХВАТ И ЭКРАНИРОВАНИЕ СТРОК ДЛЯ ИНЛАЙН-КЛИКОВ

```
const safeUsername = String(r.username).replace(/'/g, "\\');
```



Сверка имени пользователя: `if (r.username === 'admin')`

Текущий аккаунт в цикле: 'admin'

- ✗ Кнопки блокировки и удаления наглухо отключаются атрибутом `disabled`.
- 💡 Подмешивается защитный всплывающий тайтл: 'Системный root-аккаунт невозможно удалить'.

Любой другой аккаунт (ИБ-офицер)

- Кнопки активны, генерируются нативные SVG-значки и полилайны.
- ⚙ Инъект обработчиков: `window.inlineDeletePanelUser(${r.id}, '${safeUsername}')`.

ИНЖЕКТ СТРОКИ В TBODY ЧЕРЕЗ INNERHTML += ... СТИЛИЗАЦИЯ МАТОВЫХ БЭЙДЖЕЙ ЗАВЕРШЕНА

🔵 Профиль ИБ-защиты подсистемы рендеринга и пагинации:

- **Защита от XSS на уровне инлайн-атрибутов (Quote Breaking Isolation):** Имя пользователя перед сквозным инжектированием в HTML-строку таблицы принудительно экранируется регулярным выражением `.replace(/'/g, "\\')` на строке 397. Это полностью ликвидирует риск проведения атак типа XSS через внедрение одинарных кавычек в имя учетной записи сотрудника СУБД, блокируя возможность сломать синтаксис инлайнового JS-обработчика `onclick`.
- **Жесткая ИБ-блокировка системного root-аккаунта (Самоликвидация UI):** Проверки `r.username === 'admin'` на строках 350 и 358 монументально отключают кнопки деактивации и полного каскадного удаления для главного суперадминистратора. Оператор физически лишен возможности удалить или перевести в статус Blocked аккаунт `admin` на стороне веб-интерфейса.
- **Строгое приведение типов перед сетевыми транзакциями (Type Safety):** Внутри функции `inlineDeletePanelUser` идентификатор `id` принудительно конвертируется в числовой тип `Number(id)`. Если рантайм передает невалидный или деформированный строковый параметр вместо ID, снайперский ИБ-затвор `isNaN()` немедленно прерывает операцию удаления, предотвращая хаотичные `400/500` ошибки бэкенда.

--- Таблица ИБ-рисков и затворов модуля `panel_users.js` ---